Symbolic Explainable Artificial Intelligence 2026/27

027: Alien Tiles Problem

Group 18 – Artur Krystopchuk (104145), Sofia Ligeiro (116046)

1. Problem Description

The Alien Tiles consists of an $n \times n$ board with c colours, represented by $\mathbb{Z}_c = \{0, \ldots, c-1\}$. Initially, every cell has colour zero. Clicking a cell increments the colours of all cells in its row and column, $\bmod c$, with the clicked cell incremented only once. Given a target board g, the goal is to reach it using the minimum number of clicks. Although no MiniZinc model was provided on the CSPLib page for this problem, we chose to improve our initial formulation by introducing a reduced-variable model and symmetry-breaking constraints to enhance solver efficiency.

2. Model and Improvements

Baseline: The variables are $x_{ij} \in \{0, \ldots, c-1\}$, the number of clicks on square (i, j), stored in `clicks_per_square`, an $n \times n$ array with one entry per square of the board. Clicking a square c times brings every colour back to where it was, so more than $c - 1$ clicks are never needed. Every square must end with its goal colour:

$$\Big(\underbrace{\sum_{k=1}^{n} x_{ik}}_{\text{clicks in row } i} + \underbrace{\sum_{k=1}^{n} x_{kj}}_{\text{clicks in column } j} - \underbrace{x_{ij}}_{\text{counted twice}} \Big) \bmod c = g_{ij} \qquad \forall i, j,$$

and we minimize the total number of clicks $\sum_{i,j} x_{ij}$. The solver has to choose all n^2 values of x.

Reduced-Variable Formulation: The colour of a square only depends on the total clicks in its row, the total clicks in its column, and its own clicks. We store the row and column totals (mod c) in `row_clicks_mod_colours` and `column_clicks_mod_colours`, one value per row and per column:

$$R_i = \Big(\underbrace{\sum_{k=1}^{n} x_{ik}}_{\text{clicks in row } i} \Big) \bmod c, \qquad C_j = \Big(\underbrace{\sum_{k=1}^{n} x_{kj}}_{\text{clicks in column } j} \Big) \bmod c.$$

The goal condition then becomes $(R_i + C_j - x_{ij}) \bmod c = g_{ij}$, so

$$x_{ij} = (\underbrace{R_i}_{\text{row total}} + \underbrace{C_j}_{\text{column total}} - \underbrace{g_{ij}}_{\text{goal}}) \bmod c.$$

So once the solver has chosen the n row totals and the n column totals, the clicks of every square are fixed. We therefore make Gecode choose only $R_1, \ldots, R_n, C_1, \ldots, C_n$ (`int_search`): c^{2n} combinations instead of c^{n^2} (for $n = 6$, $c = 3$: 5.3×10^5 instead of 1.5×10^{17}). The model still checks that the computed clicks really add up to R_i and C_j, so no wrong totals are accepted, and both models have exactly the same solutions and the same minimum. For example, for `example_board_1.txt` ($c = 3$, goal rows 111, 100, 100), choosing $R = C = (1, 0, 0)$ gives $x_{11} = 1 + 1 - 1 = 1$ and $x_{ij} = 0$ elsewhere: one click on the top-left square.

Symmetry breaking. The solver guesses the totals R and C and rejects the guesses that fail. For the goal with rows 110, 110, 110 ($c = 3$), all rows are identical, so reordering the row totals never changes the outcome: $R = (2, 0, 1)$ and $R = (0, 1, 2)$ fail or succeed together, and checking both is wasted work. We therefore require $R_1 \leq R_2 \leq R_3$ (10 of the 27 guesses for R remain) and, as columns 1 and 2 are identical, $C_1 \leq C_2$. The solver checks 180 guesses instead of 729, and the only solution ($R = (1, 1, 1)$, $C = (0, 0, 0)$: one click on each square of column 3) still satisfies the rules. If the goal is symmetric (row j equals column j), swapping R with C also never changes the outcome, so `lex_lesseq`(R, C) keeps one of the two: compare R_1 with C_1, then R_2 with C_2, and so on, like sorting words, and R must not come after C. No solution is ever removed, only repeated guesses.

3. Execution

Requirements: Python 3.12+, MiniZinc 2.10.1 with Gecode, and the Python `minizinc` package.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python3 proj.py examples/example_board_1.txt --player model
```

The `--player` option selects the mode: `model` solves the puzzle with the CSP model and prints the number of clicks for each square and the total; `human` starts an interactive game in the terminal. If omitted, the program asks which mode to use. More details and usage information can be found in the `README.md` file.

4. Generative AI Use

Generative AI was used at Yellow Level, in accordance with the institutional guidelines, to brainstorm modelling ideas, discuss the reformulation and symmetry breaking, review code and tests, improve this report, and assist in creating the animations used in the video.

5. Video:

Video Link